

Practica 8. Problema de las casas (2) ya terminado

Garay Pérez Matthew Ilveneff

202210036@tesco.edu.mx

Tecnológico de Estudios Superiores
de Coacalco

Ingeniería en Sistemas
Computacionales

Inteligencia Artificial

Ing. Vivanco Benavides Luis Enrique

Entrega 27 de octubre de 2025

3821

Introducción

Esta práctica de programación se enmarca en el paradigma de la Programación Lógica, cuyo lenguaje más representativo es Prolog (PROgrammation en LOGique), desarrollado en la década de los 70 en Europa. La Programación Lógica se fundamenta en la Lógica de Primer Orden y difiere de los lenguajes imperativos al enfocarse en la declaración de qué se debe lograr (la lógica), en lugar de especificar cómo lograrlo (el algoritmo o control), ya que el propio motor de inferencias del lenguaje se encarga de la deducción automática. El problema resuelto, conocido como el Acertijo de Einstein o Problema de la Cebra, es un enigma clásico que sirve como un ejemplo ideal para la aplicación de este paradigma, al ser catalogado formalmente como un Problema de Satisfacción de Restricciones (CSP). En este enfoque, las 15 pistas del acertijo actúan como restricciones que limitan las combinaciones posibles de cinco atributos (nacionalidad, color, bebida, cigarrillo, mascota) en cinco casas. El código Prolog modela esta base de conocimiento mediante una colección de hechos y reglas (predicados como `member`, `vecino`, y `vecino_izq`), que definen las relaciones lógicas espaciales y de pertenencia. Al ejecutar una consulta, el sistema de Prolog utiliza la unificación y el retroceso (`backtracking`) para explorar automáticamente el espacio de estados y encontrar la única asignación de variables (la tabla final) que satisface todas las restricciones del problema.

Desarrollo

El primer paso fue establecer una representación del problema que Prolog pudiera manejar. Se utilizó una estructura declarativa donde la solución completa del acertijo se modela como una lista de cinco elementos llamada `Casitas`. Cada elemento de esta lista es, a su vez, una lista que representa una casa con sus seis atributos fijos y variable.

```

problemascasas.pl
File Edit Browse Compile Prolog Pce Help

vecino_izq([Vi, Vd | _], Vecino_Izq, Vecino_Der) :-
    (Vi = Vecino_Izq, Vd = Vecino_Der);
    vecino_izq([Vd | _], Vecino_Izq, Vecino_Der).

vecino(Lista, Info_1, Info_2) :-
    vecino_izq(Lista, Info_1, Info_2);
    vecino_izq(Lista, Info_2, Info_1).

posicacasa(Casitas):-
    Casitas=[
        [1,_,_,_,_,_],
        [2,_,_,_,_,_],
        [3,_,_,_,_,_],
        [4,_,_,_,_,_],
        [5,_,_,_,_,_]
    ],
    % 1. El noruego vive en la primera casa.
    member([1,_,noruego,_,_,_], Casitas),
    % 1. ... junto a la casa azul.
    vecino(Casitas, [_ ,_, noruego, _ ,_ ], [_ azul, _ ,_ ,_ ,_ ]),
    % 2. El que vive en la casa del centro toma leche.
    member([3,_,_,leche,_,_], Casitas),
    % 3. El inglés vive en la casa roja.
    member([_,roja,ingles,_,_,_], Casitas),
    % 4. La mascota del Sueco es un perro.
    member([_,_,sueco,_,perro,_,_], Casitas),
    % 5. El Danés bebe té.
    member([_,_,danés,te,_,_], Casitas),

```

Para modelar las restricciones que dependen de la ubicación de las casas, se definieron los predicados `vecino_izq/3` y `vecino/3`. Estos predicados son esenciales para traducir las reglas relacionales del acertijo:

`vecino_izq(Lista, Vecino_Izq, Vecino_Der)`: Define la relación de adyacencia estricta (inmediatamente a la izquierda). Es crucial para restricciones como "La casa verde está inmediatamente a la izquierda de la casa blanca."

`vecino(Lista, Info_1, Info_2)`: Define la relación de adyacencia simple (uno al lado del otro, en cualquier orden). Se usa para reglas simétricas, como "El noruego vive junto a la casa azul" o "El que fuma Blend vive junto al que tiene gatos."

Estos predicados permiten que el código maneje las restricciones espaciales de manera declarativa sin depender de índices numéricos explícitos más allá de las posiciones iniciales (Casa 1, Casa 3).

```

problemascasas.pl
File Edit Browse Compile Prolog Pce Help

problemascasas.pl

% 1. El noruego vive en la primera casa.
member([1,noruego,_,_,_],Casitas),
% 1. ... junto a la casa azul.
vecino(Casitas,[__,noruego,_,_],[__,azul,_,_,_]),

% 2. El que vive en la casa del centro toma leche.
member([3,_,leche,_,_],Casitas),

% 3. El inglés vive en la casa roja.
member([_,roja,ingles,_,_],Casitas),

% 4. La mascota del Sueco es un perro.
member([_,sueco,_,perro,_,_],Casitas),

% 5. El Danés bebe té.
member([_,danés,te,_,_],Casitas),

% 6. La casa verde es a la izquierda de la casa blanca.
vecino_izq(Casitas,[__,verde,_,_,_],[__,blanca,_,_,_]),

% 7. El de la casa verde toma café.
member([_,verde,_,café,_,_],Casitas),

% 8. El que fuma PallMall cria pájaros.
member([_,_,pajaros,pallmall,_,_],Casitas),

% 9. El de la casa amarilla fuma Dunhill.
member([_,amarilla,_,_,dunhill,_,_],Casitas),

% 10. El que fuma Blend vive junto al que tiene gatos.
vecino(Casitas,[__,_,gatos,_,_],[__,_,_,blend,_,_]),

% 11. El que tiene caballos vive junto al que fuma Dunhill.
vecino(Casitas,[__,_,_,caballos,_,_],[__,_,_,_,dunhill,_,_]),

% 12. El que fuma BlueMaster bebe cerveza.
member([_,_,_,cerveza,_,bluemaster,_,_],Casitas),

% 13. El alemán fuma Prince.
member([_,_,aleman,_,_,prince,_,_],Casitas),

% 14. El que fuma Blend tiene un vecino que bebe agua.
vecino(Casitas,[__,_,_,blend,_,_],[__,_,_,_,agua,_,_]),
member([_,_,_,_,_,_,_],Casitas),
member([_,_,_,_,_,_,_],Casitas),

color(Persona,Color):-posicasa(Casitas),
member([_,Color,Persona,_,_,_],Casitas).

bebida(Persona,Bebida):-posicasa(Casitas),
member([_,_,Persona,Bebida,_,_,_],Casitas).

fuma(Persona,Fuma):-posicasa(Casitas),
member([_,_,Persona,_,_,Fuma,_,_],Casitas).

mascota(Persona,Mascota):-posicasa(Casitas),
member([_,_,Persona,_,_,Mascota,_,_],Casitas).

numero(Persona,Numero):-posicasa(Casitas),
member([Numero,_,Persona,_,_,_],Casitas).

```

El núcleo de la práctica es la traducción de las 14 pistas a cláusulas lógicas dentro de `posicasa`. Cada cláusula actúa como una restricción que reduce el espacio de búsqueda de la solución.

Restricciones de Pertenencia (`member/2`): Las reglas que relacionan atributos dentro de la misma casa (ej. "El inglés vive en la casa roja") se implementan con el predicado estándar `member`, asegurando que una lista con esas propiedades esté presente en la lista `Casitas`.

Restricciones de Posición (`vecino`): Las reglas que involucran la adyacencia (ej. "El que tiene caballos vive junto al que fuma Dunhill") se implementan usando los predicados auxiliares definidos en el punto 2.

Al ejecutar la consulta `posicasa(Casitas)`, el motor de Prolog realiza una búsqueda exhaustiva utilizando unificación y retroceso (backtracking). Intenta asignar valores a las variables de cada casa y, si alguna restricción falla, automáticamente revierte la última decisión y prueba otra alternativa. Este proceso garantiza que se encuentre la única solución que satisface todas las condiciones del acertijo. El resultado de la unificación exitosa establece que el alemán reside en la casa 4 y tiene peces como mascota.

The first screenshot shows the Prolog IDE with the query `?- posicasa(Casitas).` and the first few results:

```

?- color(X,Y).
X = noruego,
Y = verde ;
X = sueco,
Y = azul ;
X = ingles,
Y = rojo ;
X = danes,
Y = amarilla ;
X = aleman.

?- fuma(X,Y).
X = noruego,
Y = pallmall ;
X = sueco,
Y = bluemaster ;
X = ingles,
Y = dunes ;
X = danes,
Y = dunhill ;
X = aleman,
Y = prince.

?- bebida(X,Y).
X = noruego,
Y = cafe ;
X = sueco,
Y = cerveza ;
X = ingles,
Y = leche ;
X = danes,
Y = te ;
X = aleman,
Y = agua.

?- mascota(X,peces).
X = ingles.

```

The second screenshot shows the final result where the German lives in house 4 and has fish as a pet:

```

?- posicasa(Casitas).
Casitas = [aleman,4,peces,aleman,aleman,aleman,aleman,aleman,aleman,aleman].

```

Conclusiones

La práctica reafirma la superioridad declarativa de Prolog al resolver el Acertijo de Einstein, un Problema de Satisfacción de Restricciones (CSP). Se constató que, al centrarse

únicamente en la declaración de las restricciones lógicas del acertijo mediante predicados como `member/2` y los auxiliares `vecino/3`, se simplificó drásticamente la tarea de programación, delegando la compleja búsqueda de soluciones y el manejo del retroceso (`backtracking`) al motor de inferencia interno de Prolog. La traducción exitosa de las 14 pistas del lenguaje natural a cláusulas lógicas validadas por el predicado `posicasa/1` demostró la capacidad del lenguaje para modelar relaciones complejas con precisión, resultando en la identificación de la solución única (El Alemán tiene peces). En última instancia, la práctica consolida el entendimiento de que la Programación Lógica es una herramienta eficiente y poderosa para el manejo de problemas basados en interrelaciones y dependencias lógicas, como las que se encuentran en campos como la Inteligencia Artificial.

Referencias Bibliográficas

- Alonso, J. A. (2006). Introducción a la programación lógica con Prolog. Departamento de Ciencias de la Computación e Inteligencia Artificial, Universidad de Sevilla.
- González, L. B., & López, J. C. (2013). El uso de Prolog en el aula: de lógica a inteligencia artificial. *Vinculos*, 10(2). Universidad Distrital.
- Marín, J. M., Rueda, C., & Valencia, J. (2017). Programación lógica: un enfoque para desarrollar aplicaciones. *Redalyc*.
- Koalite. (2016). Resolviendo el acertijo de Einstein. *Koalite Blog*.
- Muy Interesante. (2023). El acertijo de Albert Einstein que solo el 2% de la población puede resolver.

